

Image Processing and Machine Vision

Assignment 02: Fitting and Alignment

October, 2025

[GitHub Repository](#)

220197E Gunathilaka K. L.

Q1. LoG blob detection on sunflowers

Sunflower centers were detected using a Laplacian-of-Gaussian (LoG) scale space with non-maximum suppression across scale and space. A range $\sigma \in [1, 15]$ (25 scales) was used, with a response threshold of 1.3 and an overlap suppression of 0.5. Estimated radius were mapped by $r = \sqrt{2} \sigma$.

```

1 sigmas = np.linspace(1, 15, 25)
2 H, W = gray.shape
3 scale_space = np.zeros((len(sigmals), H, W), np.float32)
4 for i, s in enumerate(sigmals):
5     g = cv.GaussianBlur(gray, (0,0), s)
6     L = cv.Laplacian(g, cv.CV_32F)
7     scale_space[i] = (s**2) * np.abs(L) # scale-normalized
8 # 3D local-max + threshold + overlap pruning ...

```

Listing 1: LoG scale space with scale-normalized Laplacian.

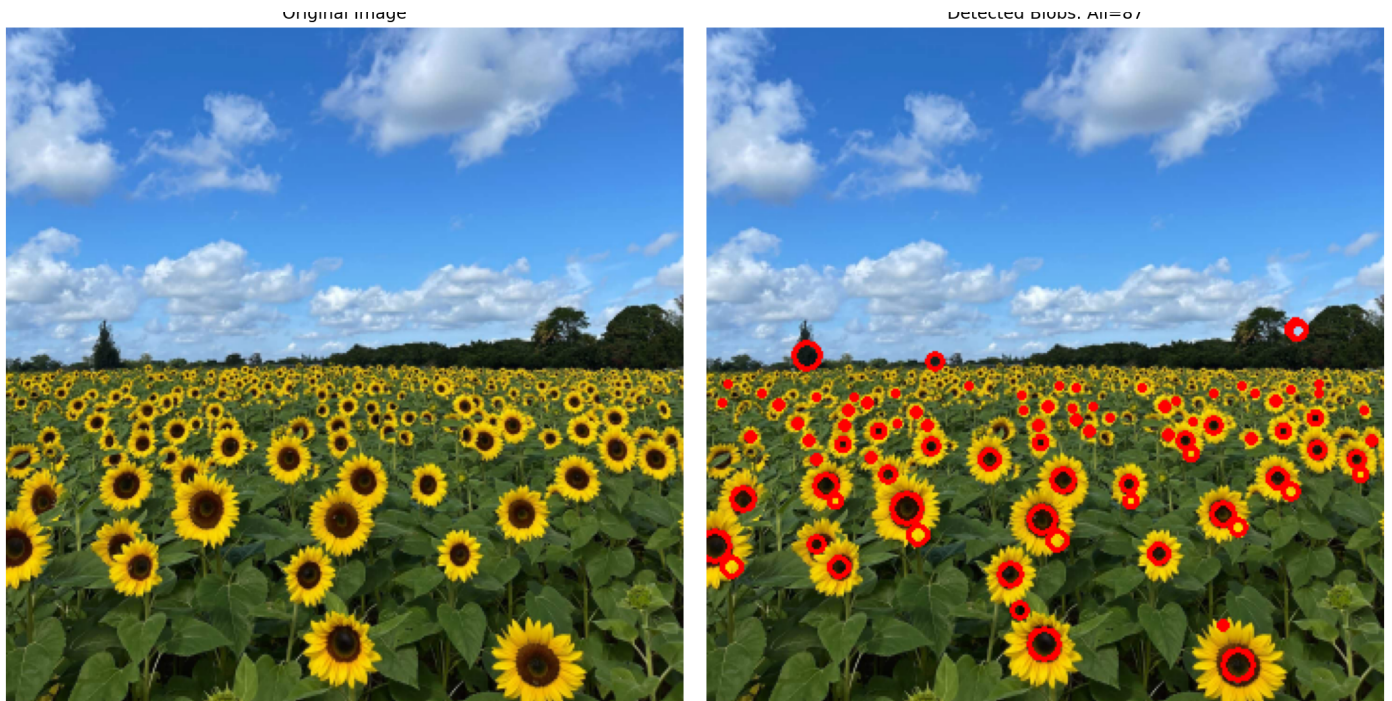


Figure 1: Original image (left) and detected blob centers (right).

Top 5 largest blobs: (1) $C = (106, 255)$, $r = 8.49$, $\sigma = 6.00$, response 0.363; (2) $C = (4, 275)$, $r = 8.49$, $\sigma = 6.00$, 0.288; (3) $C = (179, 327)$, $r = 8.49$, $\sigma = 6.00$, 0.345; (4) $C = (282, 338)$, $r = 8.49$, $\sigma = 6.00$, 0.308; (5) $C = (53, 174)$, $r = 7.74$, $\sigma = 5.47$, 0.301.

Q2. RANSAC fitting

A line was fitted by RANSAC with the normalized implicit model $ax + by + d = 0$ ($\| [a, b] \| = 1$). Inliers were selected by normal distance (< 0.5). After removing these points, a circle was fitted to the remnant via RANSAC using radial error (< 0.5).

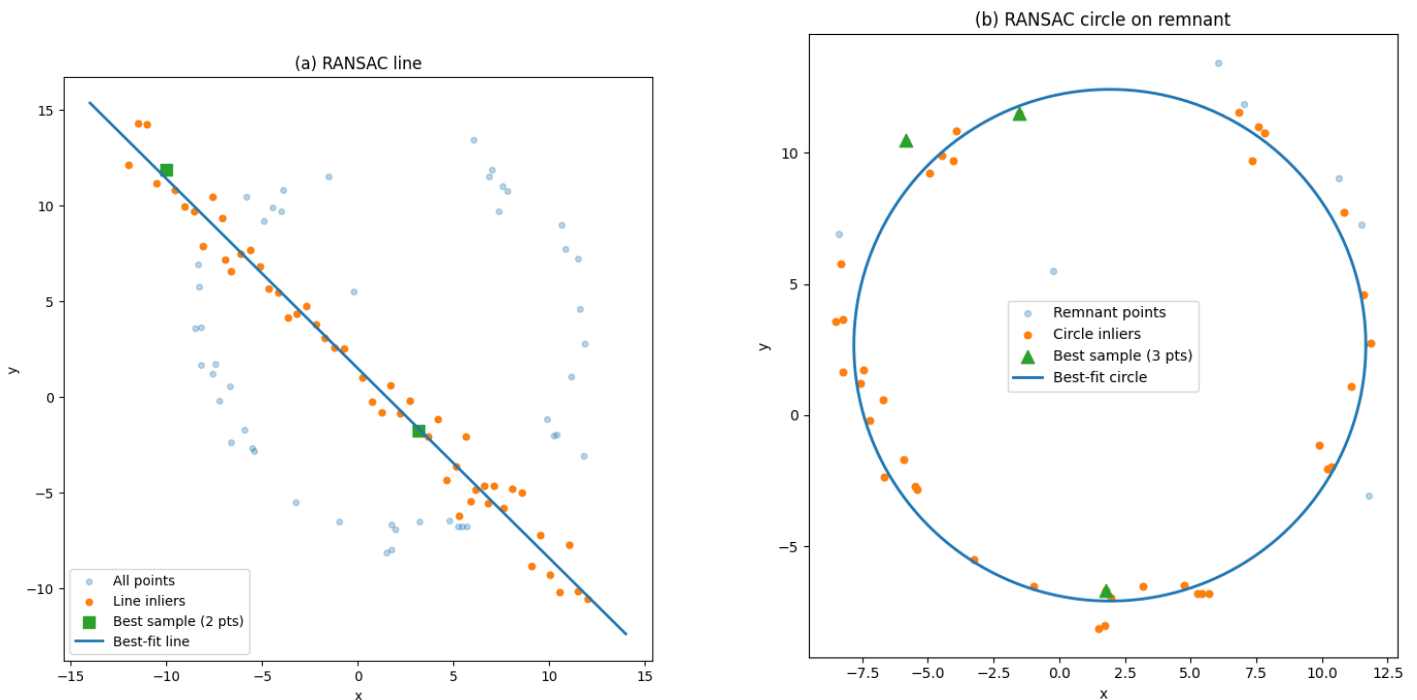
Why not fit the circle first? As many line points sit near some radius R and pass the radial-error test, and near-collinear samples yield huge-radius circles that falsely gain high consensus. The line model has a stable distance and larger true support, so fitting the line first removes the dominant structure and makes the subsequent circle fit reliable.

```

1 def fit_line_tls(P):
2     x, y = P[:,0], P[:,1]
3     X = np.column_stack([x, y, np.ones_like(x)])
4     _, _, Vt = np.linalg.svd(X)
5     a, b, d = Vt[-1]; n = np.hypot(a,b)
6     return a/n, b/n, d/n
7
8 def line_distance(P, a,b,d):
9     return np.abs(a*P[:,0] + b*P[:,1] + d)
10
11 def circle_from_points(p,q,r):
12     # closed-form circle through 3 non-collinear points -> (x0,y0,R)
13     ...
14
15 def radial_error(P, x0,y0,R):
16     return np.abs(np.hypot(P[:,0]-x0, P[:,1]-y0) - R)

```

Listing 2: Custom RANSAC for line and circle.



(a) RANSAC line: best sample, inliers, and TLS fit.

(b) RANSAC circle on the line-subtracted remnant.

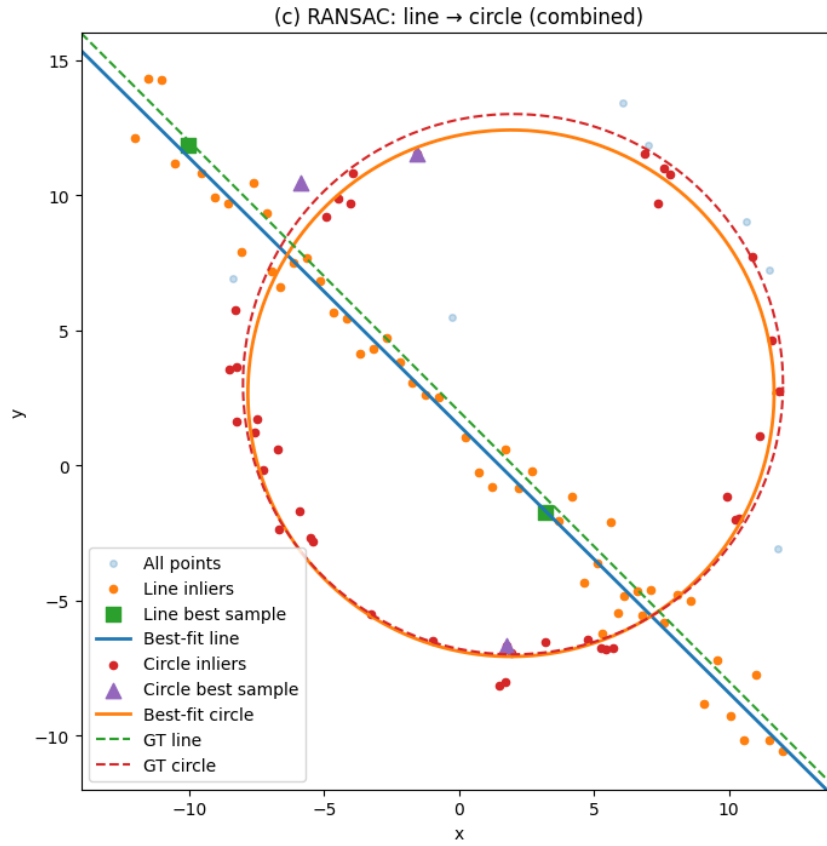


Figure 3: Combined result: final line and circle fits.

Q3. Planar homography

A planar homography was estimated from four point correspondences using the Direct Linear Transform (DLT) solved by SVD and normalized to $H_{33} = 1$. Backward warping with bilinear sampling was applied to project the source onto the destination plane.

```

1 def compute_homography(src_pts, dst_pts):
2     A = []
3     for (x,y), (u,v) in zip(src_pts, dst_pts):
4         A += [[-x,-y,-1, 0, 0, 0, x*u, y*u, u],
5              [ 0, 0, 0,-x,-y,-1, x*v, y*v, v]]
6     _, _, Vt = np.linalg.svd(np.asarray(A))
7     H = Vt[-1].reshape(3,3); return H / H[2,2]
8
9 H = compute_homography(src_corners, dst_quadrilateral)
10 warped = cv.warpPerspective(src, H, (Wout, Hout)) % backward, bilinear

```

Listing 3: DLT homography and warping.



Figure 4: Source image (left), target plane (center), and warped result (right).

The blank billboard provides a near-ideal plane, producing a perspective-consistent placement of the poster.

Q4. SIFT matching + RANSAC homography + stitching

Keypoints and descriptors were obtained with SIFT, then matched using BFMatcher (KNN, $k = 2$) with Lowe's ratio test (0.75). A homography was estimated via custom RANSAC+DLT using reprojection error (< 3 px). The final panorama was created by translating the canvas to positive coordinates, warping the source with the estimated H , and blending with simple masking.

```

1 sift = cv.SIFT_create()
2 kp1, des1 = sift.detectAndCompute(img1_gray, None)
3 kp5, des5 = sift.detectAndCompute(img5_gray, None)
4
5 matches = cv.BFMatcher().knnMatch(des1, des5, k=2)
6 good = [m for m,n in matches if m.distance < 0.75*n.distance]
7
8 pts1 = np.float32([kp1[m.queryIdx].pt for m in good])
9 pts5 = np.float32([kp5[m.trainIdx].pt for m in good])
10
11 H, inliers = ransac_homography(pts1, pts5, reproj_thresh=3.0)
12 stitched = warp_and_blend(img1, img5, H) # translate canvas, mask blend

```

Listing 4: SIFT + ratio test + RANSAC homography.



Figure 5: Top: input images. Bottom-left: SIFT matches. Bottom-right: stitched panorama.